Name :SR. Kaarthika Reg.No: 192412294 Course Code :CSA1706

1. AI-Powered Drone Route Planning in a Flood-Affected Region

i. Greedy Best-First Search (GBFS)

Working:

- Greedy Best-First Search selects the next path based only on the **heuristic value h(n)** (estimated straight-line distance to the destination).
- It always chooses the node that appears closest to the goal without considering the travel cost already incurred.

Behavior in this Scenario:

- The drone quickly moves toward the destination using the shortest estimated distance.
- It ignores varying terrain costs and weather conditions.
- If a road becomes blocked, it must discard that path and search for another.

Strengths:

- Very fast in emergencies.
- Requires less memory than many search algorithms.
- Suitable when a quick decision is more important than the best route.

Weaknesses under Uncertainty:

- Does not guarantee the shortest or safest path.
- May choose risky or expensive routes.
- Frequent rerouting occurs when blocked paths are discovered.
- Performance depends heavily on heuristic accuracy.

ii. A* Search

Working:
A* evaluates each node using

[
f(n)=g(n)+h(n)
]

where:

- **g(n)** = Actual cost from start to current node
- **h(n)** = Estimated cost from current node to goal
- **f(n)** = Total estimated cost

Behavior in this Scenario:

- Considers both travel cost and estimated remaining distance.
- Avoids routes with high terrain or weather costs.
- Recalculates the best available path when obstacles appear.

Advantages:

- Produces the optimal route if the heuristic is admissible.
- Balances speed and path quality.
- Handles variable movement costs effectively.
- Safer for emergency medicine delivery.

Disadvantages:

- Requires more computation and memory than Greedy Search.
- Dynamic changes may require repeated replanning.

iii. Impact of Heuristic Accuracy

Heuristic Accuracy	**Greedy Best-First Search**	**A* Search**
Highly Accurate	Very fast and usually finds a good path	Finds optimal path efficiently
Moderately Accurate	May choose suboptimal routes	Still finds optimal solution but explores more nodes
Poor/Inaccurate	Easily misled into wrong directions	Performance decreases but usually still reaches the optimal path if heuristic is admissible

Observation:

- Greedy Search depends almost entirely on the heuristic.
- A* is more robust because it also considers the accumulated cost.

iv. Effect of Dynamic Changes (Blocked Paths)

Greedy Best-First Search

- Continues toward the goal until a blocked path is encountered.
- Must backtrack and search again.
- Can waste time exploring unsuitable paths.

*A Search**

- Updates costs and searches for the next best route.

- Better adapts to changing environments.
- May require additional computation but usually finds a safer alternative.

Comparison:

Aspect	Greedy Search	A* Search
Response to blocked paths	Frequent backtracking	Efficient replanning
Adaptability	Low	High
Route Quality	Often poor	High
Reliability	Lower	Higher

v. Recommended Algorithm

Recommended Algorithm: A* Search

Justification:

- **Real-Time Constraints:** Although slightly slower than Greedy Search, A* still provides efficient route planning suitable for emergency situations.
- **Optimality:** Finds the minimum-cost route by considering both actual travel cost and heuristic estimate.
- **Safety:** Avoids dangerous or expensive paths caused by floods, rough terrain, or bad weather.
- **Dynamic Environment:** Adapts better to blocked roads and changing movement costs through replanning.
- **Reliability:** More dependable for delivering life-saving medicines.

Conclusion:
For AI-powered drones operating in flood-affected regions, *A Search** is the most suitable algorithm because it balances **speed, optimality, adaptability, and safety**. While **Greedy Best-First Search** is faster, it may select unsafe or inefficient routes, making it less reliable in dynamic and uncertain environments.

2. Smart City Traffic Signal Optimization

Problem Formulation

The smart city traffic control system aims to optimize traffic signal timings at hundreds of intersections.

Objective Function:
Minimize:

- Total vehicle waiting time
- Fuel consumption

- Traffic congestion

Constraints:

- Traffic flow changes continuously.
- Signals at neighboring intersections are interdependent.
- Emergency vehicles may require priority.
- Pedestrian crossing times must be maintained.

Why Traditional Search is Inefficient?

Traditional search algorithms (BFS, DFS, Uniform Cost Search) are not suitable because:

- The search space is extremely large due to thousands of possible signal timing combinations.
- Traffic conditions change dynamically.
- Searching every possible solution requires excessive computation time.
- Real-time decision-making becomes impossible.

i. Hill Climbing Algorithm

Working

Hill Climbing is a **local search algorithm**.

1. Start with an initial traffic signal timing.
2. Evaluate its performance (waiting time, congestion, fuel usage).
3. Generate neighboring solutions by slightly changing signal timings.
4. Move to the neighbor with the best improvement.
5. Repeat until no better neighboring solution exists.

Example

Current signal timing:

- Green = 40 sec
- Red = 30 sec

Neighboring solutions:

- Green = 45 sec
- Green = 35 sec

The algorithm selects whichever reduces congestion the most.

Advantages

- Simple to implement.

- Fast execution.
- Requires little memory.
- Suitable for quick optimization.

Limitations

- May stop before finding the best solution.
- Cannot escape poor local solutions.
- Performance depends on the starting point.

ii. Local Maxima, Plateaus, and Ridges

1. Local Maximum

A solution better than all nearby solutions but not the global best.

Traffic Example:
Changing one intersection improves traffic locally, but city-wide congestion remains high.

Problem:
The algorithm stops although a much better overall solution exists.

2. Plateau

A large flat region where neighboring solutions have nearly identical performance.

Traffic Example:
Changing signal timing from 40 sec to 41 sec or 42 sec produces almost no improvement.

Problem:
The algorithm cannot determine which direction to move.

3. Ridge

The best solution requires multiple coordinated changes simultaneously.

Traffic Example:
Improving traffic requires adjusting signals at several connected intersections together.

Changing only one signal shows no improvement.

Problem:
Hill Climbing cannot move because every individual change appears worse.

iii. Simulated Annealing and Genetic Algorithm

A. Simulated Annealing

Working:

- Similar to Hill Climbing.
- Occasionally accepts worse solutions with a certain probability.

- As time progresses, this probability gradually decreases.

Advantages

- Escapes local maxima.
- Explores many regions of the search space.
- Suitable for dynamic optimization.

Limitation

- Requires careful tuning of the cooling schedule.
- May converge slowly.

B. Genetic Algorithm (GA)

Working

1. Generate many different traffic signal plans.
2. Evaluate each plan using a fitness function.
3. Select the best plans.
4. Apply crossover to combine good solutions.
5. Apply mutation to introduce diversity.
6. Repeat until an optimal solution is found.

Advantages

- Searches many solutions simultaneously.
- Escapes local maxima.
- Handles large and complex search spaces.
- Easily adapts to changing traffic conditions.

Limitation

- Requires more computational resources.
- Parameter tuning is necessary.

iv. Recommended Approach

Recommended Algorithm: Genetic Algorithm (GA)

Justification

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Large Search Space	Poor	Good	Excellent
Avoid Local Optima	No	Yes	Yes

Criteria	Hill Climbing	Simulated Annealing	Genetic Algorithm
Dynamic Traffic	Moderate	Good	Excellent
Scalability	Moderate	Good	Excellent
Global Optimization	No	Good	Excellent
Real-Time Adaptability	Moderate	Good	Excellent

Conclusion

For optimizing traffic signals in a smart city, the **Genetic Algorithm** is the most suitable approach because it:

- Efficiently handles extremely large search spaces.
- Avoids local maxima through crossover and mutation.
- Optimizes multiple objectives simultaneously (waiting time, fuel consumption, congestion).
- Adapts well to continuously changing traffic conditions.
- Provides scalable performance for hundreds of intersections.

3. Autonomous Mars Rover – Online Search Agent

i. Why the Problem Requires an Online Search Agent Instead of Offline Search

An **offline search agent** assumes the complete environment is known before planning a path. This is not possible on Mars because the rover has **no complete map** and discovers obstacles only while moving.

An **online search agent** is suitable because it:

- Explores unknown terrain while moving.
- Makes decisions using current sensor information.
- Updates its path as new obstacles are detected.
- Adapts to unexpected situations in real time.

Conclusion: Since the rover learns about the environment during exploration, **online search** is essential.

ii. Challenges of Partial Observability and Dynamic Environments

1. Partial Observability

The rover's sensors can only detect nearby objects.

Challenges:

- Unknown locations of rocks and craters.
- Limited visibility of the terrain.
- Hidden obstacles may suddenly appear.
- Decisions must be made with incomplete information.

2. Dynamic Environment

Although Mars changes slowly, the rover may encounter:

- Dust storms reducing visibility.
- Loose rocks blocking paths.
- Slippery sand affecting movement.
- Sensor errors or communication delays.

Challenges:

- Previously safe paths may become unsafe.
- The rover must frequently update its route.
- Continuous replanning is required.

iii. How the Rover Builds and Updates Its Internal Model

The rover maintains an **internal map** of the environment.

Working

1. Start with little or no knowledge.
2. Move to a nearby location.
3. Use sensors to detect:
    - Rocks
    - Craters
    - Safe paths
    - Sample locations
4. Store this information in memory.
5. Update the internal map.
6. Plan the next movement using the updated information.
7. Repeat until the mission is complete.

Benefits

- Learns the environment gradually.

- Avoids revisiting dangerous locations.
- Improves navigation accuracy over time.
- Supports safer and more efficient exploration.

iv. Comparison of Online Search with Uninformed and Informed Search

Feature	**Uninformed Search**	**Informed Search**	**Online Search**
Environment Knowledge	Complete	Complete	Partial/Unknown
Uses Heuristic	No	Yes	May use heuristic
Real-Time Decision	No	Limited	Yes
Adaptability	Low	Moderate	High
Suitable for Unknown Terrain	No	No	Yes
Updates Knowledge While Moving	No	No	Yes

Observation:

- **Uninformed Search** (BFS, DFS) requires a known environment.
- **Informed Search** (A*) uses heuristics but still assumes a map is available.
- **Online Search** continuously learns and adapts, making it ideal for unknown environments.

v. Most Suitable Approach

Recommended Approach: Online Search Agent

Justification

Uncertainty

- Works effectively with incomplete maps.
- Makes decisions using available sensor data.

Adaptability

- Updates the path whenever new hazards are detected.
- Continuously improves its internal model.

Safety

- Avoids newly discovered obstacles.
- Reduces the risk of collisions and mission failure.

Real-Time Operation

- Does not wait for complete information before acting.
- Suitable for autonomous exploration where immediate decisions are necessary.

Conclusion

An **Online Search Agent** is the most suitable approach for the Mars rover because it:

- Operates effectively in unknown and partially observable environments.
- Continuously builds and updates an internal model.
- Adapts to changing conditions in real time.
- Ensures safe and efficient navigation while collecting scientific samples.

4. AI-Based University Exam Timetabling (CSP)

Problem Formulation

The exam timetable is modeled as a **Constraint Satisfaction Problem (CSP)** where exams are assigned time slots and halls while satisfying all constraints.

i. Variables, Domains, and Constraints

Variables:

- Exams/Courses (C1, C2, C3, ...)

Domains:

- Available time slots (T1, T2, T3)
- Exam halls (H1, H2, H3)

Constraints:

- No student has overlapping exams.
- Hall capacity must not be exceeded.
- Some subjects must be scheduled before others.
- Faculty must be available.
- Special accommodation must be provided.

ii. Backtracking Search

- Assign a time slot and hall to each exam.
- Check if constraints are satisfied.
- If a conflict occurs, **backtrack** and try another assignment.
- Continue until a valid timetable is found.

Advantage: Finds a valid solution if one exists.

iii. Constraint Propagation (Forward Checking)

- Removes invalid choices after each assignment.
- Detects conflicts early.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.

iv. Real-World Challenges & Best Strategy

Challenges:

- Large number of students and courses.
- Limited exam halls.
- Faculty availability.
- Last-minute schedule changes.

Best Strategy:
Use **Backtracking with Forward Checking** because it:

- Reduces the search space.
- Detects conflicts early.
- Generates efficient and scalable timetables.

Conclusion

Backtracking + Forward Checking is the best approach for exam scheduling as it efficiently handles multiple constraints and scales well for large universities.

5. Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

- **Minimax** models the game as a two-player game.
- The AI (**MAX**) tries to maximize its score, while the opponent (**MIN**) tries to minimize it.
- The AI selects the move that gives the best possible outcome assuming the opponent plays optimally.

ii. Computational Challenges

- The game tree has a very large number of possible moves.
- Searching every possible move is time-consuming and requires high memory.
- Real-time games need decisions within a few seconds.

iii. Alpha-Beta Pruning

- **Alpha-Beta Pruning** improves Minimax by skipping branches that cannot affect the final decision.
- It reduces the number of nodes evaluated without changing the optimal result.
- This makes the AI faster and suitable for real-time games.

iv. Evaluation Function

The evaluation function scores each game state based on factors such as:

- Number of units remaining
- Health of units
- Resources collected
- Territory controlled
- Distance from victory

A higher score indicates a better position for the AI.

v. Best Strategy

Recommendation: Use **Minimax with Alpha-Beta Pruning**.

Justification:

- Produces optimal decisions.
- Reduces unnecessary computations.
- Handles large game trees efficiently.
- Meets real-time performance requirements while maintaining strong gameplay.

Conclusion

Minimax with Alpha-Beta Pruning is the most suitable approach because it balances **optimal decision-making, computational efficiency, and real-time performance**.